

HW1 Report

1. Project Overview

Your product (model) should be able to do the following:

- It takes as input *several* images of the supermarket bills, plus one user query (in the form of text).
- There are two possible queries that the users are interested to know:
 1. Query 1: How much money did I spend in total for these bills?
 2. Query 2: How much would I have had to pay without the discount?
- The model should have the capacity to reject irrelevant queries.

2. Initial Approach: Direct Query

2.1 Design Concept

The initial implementation used a straightforward approach. The code sent all receipt images to the Gemini model with instructions to calculate both required totals in one operation. The system prompt asked the model to "analyze these receipts and tell me both the total amount spent and what the original price would have been without any discounts."

2.2 Implementation Results

Query 1 consistently returned accurate results (1974.30 matching the expected 1974.30). However, Query 2 showed significant errors, typically around 46.67 off from the ground truth. The main issue was that the model missed various discount amounts, especially small rounding figures and percentage-based discounts.

2.3 Problem Analysis

The direct approach failed because the model couldn't reliably extract all negative numbers representing discounts. It frequently overlooked the small rounding

amounts that appear under "SUBTOTAL" sections and didn't fully capture complex discount patterns like "Buy X Save Y" promotions.

3. Improved Approach: Structured Data Extraction

3.1 Design Concept

The system was redesigned with a two-phase approach. First, the code instructed the model to extract all numerical data from receipts in a structured format. Then, separate Python functions processed this data to calculate the required totals.

The implementation asked the model to "extract all numbers from these receipts including final totals and any amounts with minus signs, returning them in a structured format." The response was then parsed, and calculations were performed locally.

3.2 Implementation Results

Query 1 remained accurate at 1974.30. Query 2 showed improvement with errors reduced to approximately 15.90, but this still exceeded the assignment's $\pm \$2$ requirement.

3.3 Identified Issues

Several problems persisted. The model's responses sometimes didn't follow the expected format, causing parsing failures. Some discount amounts were still missed, and processing multiple images simultaneously seemed to overwhelm the model's ability to track all details accurately.

4. Key Breakthrough: Simplified Step-by-Step Processing

4.1 Source of Inspiration

Discussion with classmates revealed that processing receipts individually yielded better results than batch processing. Their experience showed that simpler instructions with clearer formatting requirements produced more reliable outputs.

4.2 Final Solution Design

The final implementation adopted a receipt-by-receipt approach. For each image, the code asked for just two pieces of information: the total paid and the sum of all discount amounts. The instruction specified "extract two numbers from this receipt: 1) the final total paid (largest number at bottom), and 2) the sum of all discount amounts (numbers with minus signs). Return in format: total=X discount=Y"

Python functions then aggregated these values across all receipts. The calculation for Query 2 simply added each receipt's total to its corresponding discount sum.

4.3 Why This Solution Works

The simplified approach worked because it gave the model a clear, focused task. Processing receipts individually prevented confusion between different bills. The text-based response format avoided parsing complications that occurred with more complex structures. Most importantly, separating data extraction from calculation allowed each component to do what it does best: the model reads and extracts information, while Python performs precise mathematical operations.

4.4 Final Results

The final system achieved excellent accuracy. Query 1 returned exactly 1974.30. Query 2 returned 2348.02, differing from the expected 2348.20 by only 0.18, well within the required $\pm \$2$ tolerance. All seven test receipts were processed successfully.

5. Learning and Reflection

5.1 Technical Insights

The project demonstrated that breaking complex tasks into simpler steps significantly improves accuracy in AI systems. Well-designed, specific instructions produce better results than comprehensive but ambiguous ones. The experience also highlighted the importance of using each tool appropriately—leveraging the model's visual recognition capabilities while relying on traditional programming for precise calculations.

5.2 Development Experience

The iterative development process, where each failure informed the next attempt, proved valuable. Collaborating with classmates provided practical insights that accelerated problem-solving. Ultimately, choosing straightforward, reliable solutions over complex ones yielded the best results.

6. Conclusion

Through multiple development iterations, the project arrived at an effective solution balancing accuracy and simplicity. The key achievement was implementing a step-by-step approach that processes receipts individually with clear instructions. This system not only meets the assignment requirements but could potentially handle new receipts in practical applications. The experience underscored fundamental principles of AI system design: divide complex problems into manageable parts and allocate tasks according to each component's strengths.